

Hardware-Accelerated AI Systolic Array Coprocessor: Architecture and Silicon Bring-up

[Peter D.]

August 26, 2026

Introduction

The exponential growth of artificial intelligence and machine learning has driven a fundamental shift in computer architecture. General-purpose processors are increasingly bottlenecked by memory bandwidth and Von Neumann architecture limitations when evaluating deep neural networks. To address this, specialized hardware accelerators such as Systolic Arrays have become the industry standard for high-throughput matrix multiplication, heavily utilized in architectures like Google’s Tensor Processing Unit (TPU).

In this project, we architect, implement, and verify a parameterized $N \times N$ Systolic Array accelerator optimized for quantized (INT8/UINT8) neural network inference. The hardware IP is deployed as a memory-mapped and stream-driven coprocessor on an Intel Agilex 5 System-on-Chip (SoC) using the Terasic DE25-Standard development board.

1 Hardware Design: The Compute Core

1.1 Mathematical Model

The fundamental mathematical operation performed by the systolic array is matrix multiplication, $C = A \times B$. For an $N \times N$ matrix, each element $C_{i,j}$ is computed as the dot product of the i -th row of A and the j -th column of B :

$$C_{i,j} = \sum_{k=1}^N A_{i,k} \cdot B_{k,j} \quad (1)$$

Within the systolic architecture, this summation is distributed both spatially and temporally. At any given clock cycle t , a Processing Element (PE) at grid coordinate (i, j) performs a Multiply-Accumulate (MAC) operation and forwards the operands to adjacent PEs for the next cycle:

$$C_{i,j}^{(t)} = C_{i,j}^{(t-1)} + \left(A_{i,k}^{(t)} \times B_{k,j}^{(t)} \right) \quad (2)$$

$$A_{i,k+1}^{(t+1)} = A_{i,k}^{(t)} \quad (\text{Forward right}) \quad (3)$$

$$B_{k+1,j}^{(t+1)} = B_{k,j}^{(t)} \quad (\text{Forward down}) \quad (4)$$

1.2 Key Entity Parameters

To ensure reusability across different Intel SoC variants, the core hardware modules are strictly parameterized using VHDL generics. Table 1 outlines the primary configuration parameters for the top-level `systolic_array` entity.

Parameter	Description
<code>N</code>	Array dimension (number of rows/columns). Defines the grid size.
<code>DATA_WIDTH</code>	Bit-width of input matrix elements A and B (e.g., 8 for INT8 workloads).
<code>ACC_WIDTH</code>	Bit-width of the internal MAC accumulators (typically 32 to prevent overflow).
<code>SIGNED_ARITH</code>	Boolean flag enabling signed (two’s complement) vs. unsigned arithmetic.

Table 1: Key Entity Parameters for the Systolic Array Core

1.3 Parameterized Architecture

The core computational engine was developed in VHDL-2008. To ensure scalability across different FPGA tiers, the IP relies on VHDL generics, specifically a parameter N , allowing the array to synthesize as any $N \times N$ grid of Processing Elements (PEs) without code modification.

For the Agilex 5 target, the array was instantiated as a 4×4 core ($N = 4$). This dimension was deliberately chosen to match the bandwidth of a 32-bit Avalon-ST DMA bus, which provides exactly four 8-bit operands per clock cycle, perfectly saturating the array inputs.

1.4 Agilex 5 AI Tensor Blocks

The Agilex 5 architecture introduces highly optimized DSP primitives known as AI Tensor Blocks, specifically designed to accelerate low-precision (INT8/FP16) matrix arithmetic. The VHDL MAC (Multiply-Accumulate) operations within our Processing Elements were carefully coded using standard operators to ensure Quartus Prime Pro automatically inferred and mapped the logic directly into these physical AI Tensor Blocks, thereby maximizing F_{MAX} and minimizing logic utilization.

2 Control and Data Flow

A defining characteristic of Systolic Arrays is the requirement for diagonal data skewing. Data elements must enter the array staggered by exactly one clock cycle per row/column to ensure correct operand pairing within the PEs.

Rather than offloading this complex timing requirement to software, we engineered a custom hardware **Mealy Finite State Machine (FSM)**. This FSM autonomously handles:

1. **Data Ingestion:** Managing Avalon-ST ‘ready/valid’ handshakes to ingest unskewed matrices from the upstream Scatter-Gather DMA.
2. **Diagonal Skewing:** Applying the exact cycle delays required before feeding the systolic grid.
3. **Result Serialization:** Capturing the massive 512-bit parallel result vector (sixteen 32-bit accumulators) and serializing it word-by-word into the output FIFO for the downstream DMA.

3 System-on-Chip (SoC) Integration

The IP was packaged using Intel Platform Designer (Qsys). It exposes two primary AMBA-style interfaces to the SoC interconnect:

- **Avalon-ST (Streaming):** Dual streaming interfaces (sink and source) are connected directly to Modular Scatter-Gather DMAs (mSGDMA), providing high-speed memory access by-passing the CPU.
- **Avalon-MM (Memory-Mapped):** A lightweight CSR interface allows the host CPU to assert control signals, monitor status flags, and manage interrupts.

3.1 Memory Mapped Interface and Control Registers

The Avalon-MM interface maps the IP into the SoC address space, allowing the Cortex-A9 or JTAG master to configure the core and monitor its execution. Table 2 defines the base memory locations for the primary Control and Status Registers (CSRs).

Address Offset	Name	Mode	Description
0x00	REG_CTRL	W	Control Register (Triggers array execution)
0x04	REG_STATUS	R/W1C	Status Register (System state and flags)
0x08	REG_CONFIG	R/W	Configuration (IRQ flags, Continuous mode)
0x18	REG_FIFO	R	Internal FIFO depths and current fill levels
0x24	REG_CAPABILITY	R	Hardware configuration (N , Data Widths)

Table 2: Avalon-MM Register Memory Map

To provide granular control, the REG_STATUS register at offset 0x04 exposes individual bit-fields detailing exactly what state the hardware FSM is currently in, as described in Table 3.

Bits	Name	Mode	Description
31:7	reserved	RO	Reads as 0, writes ignored.
6	err_overflow	R/WIC	1 if an accumulator overflowed during MAC ops.
5	out_fifo_full	RO	1 if the Avalon-ST output FIFO has no space left.
4	out_fifo_empty	RO	1 if the Avalon-ST output FIFO contains no results.
3	in_fifo_ready	RO	1 if sufficient matrix data is buffered to begin.
2	irq_pending	RO	1 if an interrupt has been raised to the GIC.
1	done	R/WIC	1 when the FSM successfully serializes the output.
0	busy	RO	1 while the systolic core is actively computing.

Table 3: Bit-field mapping for the REG_STATUS memory location

4 Silicon Bring-Up and Verification

Initial functional verification was performed via ModelSim/Questa using UVM testbenches. However, the final hardware-software co-design was brought up on bare-metal silicon using Intel **System Console** and **TCL scripting**.

The TCL scripts act as a JTAG Master, orchestrating the following sequence:

1. Asserting soft-resets via the REG_CTRL CSR.
2. Writing raw Matrix A and Matrix B data directly into the SoC SDRAM.
3. Configuring the mSGDMAs with descriptor blocks pointing to the SDRAM addresses.
4. Arming the DMAs and pulsing the REG_CTRL start bit.
5. Polling the REG_STATUS register until the ‘Done’ flag is asserted.
6. Reading the resulting Matrix C from SDRAM and verifying the matrix multiplication mathematics against a golden reference.

5 Silicon Errata & Debugging

During on-silicon verification, two highly complex integration anomalies were encountered. The identification and resolution of these bugs highlighted the intersection of software scripting and physical FPGA primitives.

5.1 JTAG System Console Endianness

When injecting matrices into SDRAM via System Console's `master_write_32` commands, mathematical verification consistently failed with scrambled operands. Debugging revealed that the System Console automatically byte-swaps 32-bit words when crossing the JTAG-to-Avalon bridge.

This required implementing explicit Big-Endian packing in the TCL script to pre-scramble the data before transmission, effectively countering the hardware swap. A `swap_endian_32` software helper function was utilized to unpack the results.

5.2 M20K Block RAM: 1-Cycle FIFO Read Latency

The most severe bug manifested as a mathematically correct output that was spatially misaligned. Specifically, the array generated the row `[220, 230, 240, 210]` instead of the mathematically expected `[210, 220, 230, 240]`.

Root Cause Analysis: The `sa_fifo` component was designed under the assumption of a purely combinational First-Word Fall-Through (FWFT) architecture. However, during Synthesis and Fitting, Quartus mapped these FIFOs into physical Agilex M20K Block RAM primitives to save ALM logic resources.

M20K Block RAMs possess an inherent, unavoidable 1-clock-cycle synchronous read latency. This physical latency caused the Mealy FSM to ingest data one cycle late relative to the diagonal skewing state machine, resulting in the 4th element of the row wrapping around to the 1st position (the observed 210 shift).